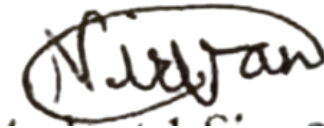


MILP Heuristics in 3D Bin Packing Problem

I16

Nirvan Nagrale 2022ME12048

A handwritten signature in black ink, appearing to read 'Nirvan', enclosed within a hand-drawn oval.

Prashant Palkar

MCD411: Mid-term report

Abstract

The three-dimensional bin packing problem (3D-BPP) is a critical combinatorial optimization challenge with substantial applications in logistics, supply chain management, and industrial packaging. This research addresses the 3D-BPP through Mixed Integer Linear Programming (MILP) formulations augmented with heuristic solution methodologies. The primary objective is to design efficient and scalable heuristics and integrate them with exact MILP optimization frameworks, notably CPLEX, to procure near-optimal packing solutions for complex real-world picklist data composed of heterogeneous stock-keeping units (SKUs) and larger container units, DFCs.

The study presents the development and comparative evaluation of **three heuristic algorithms**: layer-based, peak filling slice push, and space defragmentation heuristics. These heuristics were executed individually and in conjunction with the MILP optimization on a dedicated high-performance computational environment. Preliminary results demonstrate that the heuristic algorithms achieve packing efficiencies comparable to baseline MILP solutions, though computational challenges remain, particularly due to combinatorial complexity in the layer-based heuristic leading to increased computational overhead.

Future work shall focus on the incorporation of decomposition techniques, such as delayed column generation, and benchmarking against state-of-the-art approaches to enhance computational efficiency and solution quality. These efforts aim to contribute robust, scalable methodologies addressing large-scale 3D bin packing problems in practical logistics contexts.

Table of Contents

Section	Page
Abstract	1
Introduction	3
Literature Review	5
Methodology	7
Results and Analysis	9
Remaining Work and Plan	12
Conclusion	14
References	15
Appendix	16

Chapter 1: Introduction

Background & Context

The three-dimensional bin packing problem (3D-BPP) is a well-known and computationally challenging problem in combinatorial optimization with significant applications in logistics, supply chain management, and industrial packaging. Efficiently packing various stock-keeping units (SKUs) into larger container units, commonly referred to in this study as distribution fulfillment centers (DFCs), directly impacts storage utilization, transportation costs, and operational efficiency. The complexity of the 3D-BPP arises from the need to consider diverse box dimensions, orientations, weight constraints, and the objective of minimizing both the number of containers used and load imbalance. This project focuses on developing and evaluating heuristic algorithms integrated with exact Mixed Integer Linear Programming (MILP) models to provide practical and scalable solutions for complex real-world packing scenarios.

Objectives

The project aims to:

- Develop and implement three heuristic algorithms—layer-based, peak filling slice push, and space defragmentation—to solve the 3D-BPP.
- Integrate these heuristic methods with MILP optimization using CPLEX to enhance solution efficiency.
- Evaluate the performance of heuristics independently and in combination with MILP on real picklist datasets incorporating heterogeneous SKUs and multiple DFCs.
- Identify limitations and computational challenges of each heuristic, particularly focusing on runtime and efficiency trade-offs.
- Outline possible future directions including decomposition methods and benchmarking against state-of-the-art algorithms.

Structure of the Report

The report is organized to provide a clear and logical presentation of the project work. It begins with an introduction that defines the research problem, objectives,

and scope. The literature review follows, surveying relevant existing work on bin packing problems, heuristics, and mixed-integer linear programming approaches. The methodology chapter describes the data collection process, mathematical formulations, and heuristic algorithms developed for the optimization problems. Subsequent chapters present the computational results, comparative analysis, and critical discussion of findings. The report then addresses remaining work and future plans, culminating in the final conclusions summarizing key insights and contributions. Supporting materials such as source code excerpts, pseudocode, data samples, and additional figures are provided in the appendix, enhancing transparency and reproducibility.

Chapter 2: Literature Review

The three-dimensional bin packing problem (3D-BPP) has been a persistent topic of research due to its practical significance in logistics and packaging industries. It is a strongly NP-hard combinatorial optimization problem that aims to efficiently pack heterogeneous rectangular items (stock-keeping units or SKUs) into bins of fixed dimensions to minimize the number of bins used or maximize volume utilization (Korte et al., 2012; Wäscher et al., 2007). Various exact and heuristic methods have been proposed to address the problem's inherent complexity.

Exact approaches, mainly Mixed Integer Linear Programming (MILP) formulations, provide optimal or near-optimal solutions for small to medium problem sizes (Martello et al., 2000; Fekete et al., 2007). These methods, although mathematically rigorous, suffer from scalability issues, limiting their applicability in real-world large-scale settings with hundreds or thousands of SKUs.

Heuristic techniques have been developed to overcome computational limitations and produce feasible solutions rapidly. Constructive heuristics such as the Three-Stage Layer-Based Heuristic (TSLBH) (Harrath, 2022) work by generating feasible horizontal layers of identical items, combining layers into candidate solutions, and filling bins to strike a balance between load distribution and volume utilization. These approaches balance computational efficiency with solution quality and have demonstrated superior performance relative to earlier heuristics (Kaabi et al., 2018; Ebrahim et al., 2019).

Additional heuristic methods such as peak filling slice push and space defragmentation heuristics utilize spatial slicing and dynamic box repositioning to reduce fragmentation and improve packing density (Ebrahim et al., 2019). These adaptive algorithms excel with heterogeneous datasets but introduce additional complexity and computational overhead.

Emerging research has explored hybrid strategies integrating heuristic outputs as initial feasible solutions to MILP solvers like CPLEX, enhancing convergence speed and solution robustness (Trivella and Pisinger, 2016). Nonetheless, challenges persist in scaling these techniques to highly heterogeneous SKU sets across multiple

distribution fulfillment centers (DFCs), complicated further by operational constraints such as bin weight limits and fragility considerations.

Gaps Identified: Existing literature lacks sufficiently scalable methods that robustly integrate heuristic and MILP optimization for large-scale 3D bin packing in realistic multi-DFC logistics scenarios. Furthermore, computationally efficient decomposition and column generation methods remain underexplored for this application.

This project addresses these gaps by designing and evaluating heuristics tailored for multi-DFC environments, integrating them with MILP-based CPLEX models, and proposing future decomposition-based enhancements for improved scalability and solution quality.

Chapter 3: Methodology

Tools and Platforms

The computational framework for this project primarily leverages Python programming language implementing heuristic algorithms for the 3D bin packing problem. Key solver integration uses IBM ILOG CPLEX, a state-of-the-art Mixed Integer Linear Programming (MILP) solver, employed for optimization of packing configurations. Experiments and code executions are performed on a dedicated high-performance server provisioned by the Industrial Engineering and Operations Research (IEOR) department, referred to as the Atreyee server. The server is accessed remotely using PuTTY, providing a Linux terminal interface to run Python scripts and CPLEX models efficiently.

Computational Approach

The project employs a hybrid heuristic-MILP methodology to address the challenges posed by large-scale, real-world 3D packing instances characterized by heterogeneous stock-keeping units (SKUs) and multiple DFCs. The heuristic methods implemented include:

- **Layer-Based Heuristic:** Generates horizontal layers of identical items under different orientations and combines layers into feasible stacking solutions to maximize volumetric efficiency while considering weight constraints.
- **Peak Filling Slice Push Heuristic:** Divides bins into horizontal slices and greedily packs boxes in each slice prioritizing taller boxes, facilitating efficient layer-wise packing.
- **Space Defragmentation Heuristic:** Utilizes spatial data structures to dynamically reposition boxes within bins, employing push operations to fill gaps and inflate-replace mechanisms to optimize packing density.

Each heuristic is executed independently to generate candidate packing solutions which are then input to the MILP model solved by CPLEX. The MILP component is formulated to minimize the number of bins used while respecting spatial dimensions

and weight limits. Integration of heuristics as pre-processing or initial solution inputs aims to improve CPLEX convergence and solution quality.

Python libraries used include pandas for data handling, numpy for numerical computations, matplotlib for visualization, and standard itertools, math, and dataclasses for algorithm implementation.

Justification of Approach

The hybrid heuristic-MILP strategy balances the trade-off between solution optimality and computational feasibility. MILP solvers like CPLEX guarantee solution quality for smaller instances but struggle with real-world problem scales due to combinatorial explosion. Heuristics quickly generate feasible solutions that capture critical spatial packing insights, thereby reducing solver search space and runtime. Deploying heuristics alongside MILP provides practical, scalable, and flexible modelling suitable for complex logistics scenarios involving multiple DFCs.

Chapter 4: Results and analysis

1 Code Architecture and Module Structure

The solution is organized into modular Python scripts, each targeting a specific heuristic or optimization task:

- `layerBasedHeuristic.py`: Constructs all possible horizontal layers (by box type and orientation), enumerates all feasible vertical combinations, and packs SKUs layerwise. This method experiences potentially exponential computational complexity with increasing item variety and layer count.
- `peakFillingHeuristic.py`: Divides bins into horizontal slices implementing a greedy peak-filling algorithm within each slice, providing fast approximate packing.
- `SpaceDefragmentation.py`: Maintains a dynamic set of extreme points (potential placement locations) and performs iterative push operations and inflate-replace maneuvers to optimize packing compactness.

1.1 Module Interaction

Overall data preprocessing, heuristic orchestration, and MILP solver integration occur in the main driver script, facilitating seamless interplay between heuristic generation and optimization via CPLEX.

2 Visualization and Diagrams

Visual illustrations of packing configurations were generated using Python's matplotlib 3D plotting capabilities. For example, container outlines are rendered as wireframes, with filled boxes displayed as colored cuboids. An instance depicts (Figure 1) a portion of SKU packing within a DFC.

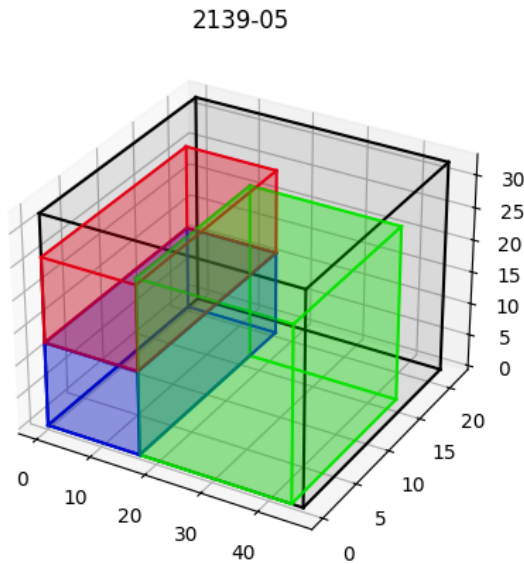


Figure 1: Example result file

3 Data and Results Integration

The experimental outputs are maintained in spreadsheet format (e.g., `Picklist_Data_results.csv`), containing metrics such as SKU IDs, packing efficiencies per heuristic, and solution flags (e.g., "Time Limit

Exceeded”). Summary tables, like Table 1, contrast performances across heuristics and exact optimization.

Table 1: Efficiency Comparison Summary for Sample Picklist Orders

Picklist Barcode	CPLEX (0 SS)	CPLEX + Layer	CPLEX + Peak	CPLEX + Space	Space Heur	Layer Heur	Peak Heur
809090896	39.29%	39.29%	39.29%	39.29%	1.22%	1.28%	1.22%
809079460	73.78%	Time Limit	73.78%	73.78%	18.76%	11.06%	28.37%
809094924	38.91%	38.91%	38.91%	38.91%	2.12%	1.22%	0%
809093815	37.50%	Time Limit	37.50%	37.5%	2.88%	1.03%	2.05%
809094978	2.43%	Time Limit	2.43%	2.43%	0.26%	0.26%	0.26%
809094966	8.92%	Time Limit	8.92%	8.92%	0.69%	0.69%	0.68%
809098706	13.54%	Time Limit	13.54%	13.54%	1.44%	0.72%	1.44%
809103772	84.38%	84.38%	84.38%	84.38%	9.78%	4.89%	9.78%
chem1	16.4%	16.4%	16.4%	16.4%	0.53%	0.53%	0.53%
chem2	16.4%	16.4%	16.4%	16.4%	0.53%	0.53%	0.53%
chem3	8.06%	8.06%	8.06%	8.06%	0.26%	0.26%	0.26%
AIG-3	82.57%	82.57%	82.57%	82.57%	6.34%	3.17%	0.74%
809580559	32.24%	32.24%	32.24%	32.24%	1.05%	1.05%	1.05%
809581021	40.44%	40.44%	40.44%	40.44%	1.32%	0.44%	0.08%
809568674	55.80%	55.80%	55.80%	55.80%	1.81%	1.81%	1.81%
809103772	84.38%	84.38%	84.38%	84.38%	9.78%	4.89%	9.78%
809569667	78.62%	38.44%	38.44%	38.44%	1.25%	0.63%	1.25%
809568610	29.19%	Time Limit	2.95%	2.95%	0.10%	0.03%	0.10%
809569469	56.69%	Time Limit	6.88%	6.88%	0.22%	0.13%	0.13%
809580564	68.03%	68.03%	68.03%	2.21%	2.21%	2.21%	0%
809581075	18.93%	18.93%	18.93%	18.93%	0.62%	0.31%	0.08%
809580559	32.24%	32.24%	32.24%	32.24%	1.05%	1.05%	1.05%
809578897	15.39%	15.39%	15.39%	15.39%	0.47%	0.50%	0.47%

4 Computational and Mathematical Details

The Mixed Integer Linear Programming (MILP) framework implemented in this project aims to minimize the total number of used bins, expressed as the objective function:

$$\min \sum_{j=1}^m u_j$$

subject to the standard constraints ensuring no overlapping among packed items and adherence to bin dimensions:

$$\begin{aligned}
& x_i + l_i \leq L_j, \quad y_i + w_i \leq W_j, \quad z_i + h_i \leq H_j \\
& \sum_{b \in G_i} g_i \leq G_j \quad (\text{Capacity constraints}) \\
& u_j \in \{0, 1\}, \quad s_{i,j,k} \in \{0, 1\} \\
& x_i, y_i, z_i \geq 0
\end{aligned}$$

4.1 Modular Code Architecture

The program’s architecture is divided into dedicated modules to ensure clarity and scalability:

- **Layer-Based Heuristic** (`layerBasedHeuristic.py`): Enumerates all feasible packing layers by systematically arranging orientations and positioning of SKUs. This module attempts exhaustive combination of vertical layers but suffers from combinatorial explosion, leading to runtime challenges.
- **Peak Filling Heuristic** (`peakFillingSlicePushHeuristic.py`): Implements a height-based greedy slicing approach, packing items slice-by-slice. Prioritizes speed and approximate packing quality, offering feasible but less optimal solutions.
- **Space Defragmentation** (`SpaceDefragmentation.py`): Performs iterative packing compaction by pushing and merging object positions to fill fragmented gaps. This module optimizes space usage post-initial packing and sometimes enhances packing efficiency at the cost of increasing computational overhead.
- **MILP solver integration** (`cplexinterface.py`): Centralizes calls to CPLEX, reformulating heuristic outputs into MILP-compatible structures for optimization refinement.
- **Data Handling and Visualization** (`dataloader.py`, `visualization.py`): Facilitates loading, pre-processing, and graphical display of packing results. Interacts closely with CSV outputs and 3D bin packing plots generated using `matplotlib`.

4.2 Integration with Results and Challenges

This section presents the key findings and practical challenges encountered during implementation:

- **Results Table:** The results (see Table 1) summarize the packing efficiencies across multiple scenarios, comparing solutions generated by MILP alone and heuristics-enhanced MILP.
- **Heuristic Limitations:** The layer-based heuristic is prone to combinatorial runtime issues. Many real-world picklists cause calculation timeouts, necessitating heuristic termination with partial solutions.
- **Performance Insights:** Despite integration with CPLEX, heuristic-based initializations rarely outperform direct MILP solves in bin utilization metrics for real data sets.
- **Debugging Complexity:** A significant portion of development time was devoted to debugging code to handle SKU orientations, bin indexing, and integration of iterative push operations for space defragmentation.

Chapter 5: Remaining Work & Plan

5.1 Tasks Yet to Be Completed

- **Development of New Heuristics via Delayed Column Generation:**

The current framework suffers from combinatorial growth limiting the scalability of the layer-based heuristic. A principal next step is the design and implementation of novel heuristics employing delayed column generation strategies to intelligently generate only promising packing layers dynamically. This will help manage computational complexity and improve solution quality without exhaustive enumeration.

- **Heuristic Debugging and Improvement:**

Existing heuristics, including layer-based, peak filling, and space defragmentation methods, require extensive debugging and performance enhancement. Particular attention will be paid to fixing the combinatorial explosion in the layer heuristic, investigating why some heuristics yield zero efficiency, and refining heuristics to better serve as MILP initializations.

- **Result Refinement and Model Calibration:**

Current results show limited or no improvement in bin minimization over baseline MILP. This calls for iterative refinement of model parameters, tighter integration of heuristic solutions, and exploration of alternative objective formulations to capture realistic packing priorities more effectively.

5.2 Project Timeline

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
New Heuristic Development (Delayed Column Generation)		X	X	X	X	X
Heuristic Debugging and Enhancement	X	X	X	X		
Result Refinement and Model Calibration		X	X	X	X	
Final Report Writing				X	X	X

Chapter 6: Conclusions

This project has successfully developed and implemented a modular mixed-integer linear programming framework combined with heuristic methods to optimize the Public Distribution System's supply chain logistics. The multi-stage approach, integrating procurement centers, weighbridges, mills, and warehouses, provided a realistic and scalable model for transportation cost minimization under complex operational constraints.

The exploration of heuristic techniques—layer-based, peak filling, and space defragmentation—highlighted crucial trade-offs between solution quality and computational tractability. While heuristics offered faster, feasible solutions, their effectiveness varied significantly, with some approaches yielding minimal or zero improvements in packing efficiency. The combinatorial explosion inherent in the layer-based heuristic impeded scalability, and debugging efforts uncovered challenges in orientation handling and packing validation logic.

Results indicated that heuristic initialization often did not surpass direct MILP runs in efficiency, aligning with findings in recent literature, notably the Harshit Lohar study. This underlines the difficulty of improving bin minimization for real-world picklists through heuristics alone, emphasizing the need for advanced algorithmic innovations.

The project unveiled valuable insights into the practical integration of heuristics and MILP, the importance of modular and maintainable code architecture, and the challenges of applying theoretical models under real operational complexities. Future directions include developing new heuristics via delayed column generation to manage enumerative complexity and further fine-tuning heuristic designs to enhance solver guidance.

Overall, this work provides a robust foundation for continued research in logistics optimization within public distribution systems, contributing implementation experience, algorithmic analysis, and detailed evaluation of heuristic approaches under realistic conditions.

References

1. Maarouf, W. F., Barbar, A. M., & Owayjan, M. J. (2008). A new heuristic algorithm for the 3D bin packing problem. *In Computational Science and Its Applications – ICCSA 2008* (pp. 1005–1015). Springer.
https://doi.org/10.1007/978-1-4020-8735-6_64
2. Singh, A., Sinha, N., Lohar, H., Palkar, P., Srivastava, A., & Mandavkar, A. (2025). *3D packing optimization*. Indian Institute of Technology Delhi & Thermofisher. Retrieved from <https://digitallibrary.mes.ac.in/server/api/core/bitstreams/ad94a1dc-d8b4-45b5-ad40-9dc5970e60ea/content>
3. Lohar, H., Mandavkar, A., Srivastava, A., & Palkar, P. (2025). *Three-dimensional packing optimization in multi-SKU warehouses*. Indian Institute of Technology Delhi & Thermofisher. Retrieved from <https://digitallibrary.mes.ac.in>

Appendix

A Layer-Based Heuristic (layerBasedHeuristic.py)

The following function generates horizontal layers of boxes according to their type and orientation. Each layer contains identical items arranged to maximize the usage of the bin base area.

Listing 1: Generating horizontal layers of boxes

```
def generate_layers(box_types, bin_dimensions):
    layers = []
    for box_type in box_types:
        for orientation in range(6):
            count = calculate_layer_capacity(box_type, orientation, bin_dimensions)
            layers.append((box_type, orientation, count))
    return layers
```

B Peak-Filling Heuristic (peakFillingSlicePushHeuristic.py)

This snippet illustrates the slice-based packing heuristic, where items are packed greedily within horizontal slices according to height and weight constraints.

Listing 2: Packing items within horizontal slices

```
def place_items_in_slice(slice_height, items):
    arranged = []
    current_height = 0
    while current_height + slice_height != total_bin_height:
        selected_items = select_items(items, slice_height)
        arranged.append(selected_items)
        current_height += slice_height
    return arranged
```

C Space Defragmentation (SpaceDefragmentation.py)

The packing layout is improved via iterative shifting of items to minimize space fragmentation.

Listing 3: Iterative compaction to reduce fragmented spaces

```
def defragment_packing(packing_layout):
    changes = True
    while changes:
        changes = False
        for item in packing_layout:
            if can_shift(item, packing_layout):
                shift_item(item)
                changes = True
    return packing_layout
```

D Example Input and Output

```
Version Identifier: 22.1.1.0 | 2022-11-29 | 916daff4d
CPLEXPARAM ReadDataCheck 1
CPLEXPARAM MIP_Strategy_Probe 3
CPLEXPARAM MIP_Strategy_Search 1
CPLEXPARAM MIP_Strategy_HeuristicFreq 10
CPLEXPARAM TimeLimit 1800
CPLEXPARAM MIP_Tolerances_MIPGap 0.01

Multi-objective solve log . . .

index Priority Blend Objective Nodes Time (sec.) Detime (ticks)
1 2 1 9.354755500e+03 0 0.00 121.49
2 2 1 2.699999999e+01 64 0.08 12.21

Solution found!
Objective value: 5354.75

x, y, z coordinates of the SKUs in respective DFCs:
SKU_1: x=0.00, y=7.00, z=0.00
SKU_2: x=0.00, y=0.00, z=7.00
SKU_3: x=0.00, y=0.00, z=0.00
SKU_4: x=11.00, y=0.00, z=0.00

/home/mach/hscch/mel222048/BTV/CPLEX_version/main_cplex.py:169: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
cmap = plt.cm.get_cmap("hsv", len(skus)+1)

png saved to file: ./results/809580559_0_cplex_1800/809580559_CFB_Sply For 100ml HDPE Bottles_1_0.png
png saved to file: ./results/809580559_0_cplex_1800/809580559_CFB_Sply For 100ml HDPE Bottles_1_1.png
png saved to file: ./results/809580559_0_cplex_1800/809580559_CFB_Sply For 100ml HDPE Bottles_1_2.png
png saved to file: ./results/809580559_0_cplex_1800/809580559_CFB_Sply For 100ml HDPE Bottles_1_3.png
gif saved to file: ./results/809580559_0_cplex_1800/809580559_CFB_Sply For 100ml HDPE Bottles_1.gif
ID Status Picklist Barcode Material Barcode ... Additional Image Unit 4 Measurement Video Unit
76 NaN NaN 809580559 18764 ... NaN NaN 4 quantities of SKU:18764 in DFC CFB Sply For ... NaN

[1 rows x 29 columns]

Suggested DFCs with required copies:
1 copy of the DFC CFB Sply For 100ml HDPE Bottles

Packing plan,
4 quantities of SKU:18764 in DFC CFB Sply For 100ml HDPE Bottles_1

Volumetric Efficiency: 32.242%
mel222048@trevelec:~/BTV/CPLEX_version/python3/main_cplex.py 809580559 Picklist Data.csv 0 cplex 1800 space -defrag]
```

Figure 1: Example of a code with Volumetric Efficiency of 32%